

Crosshair Module Redesign Specification

CRITICAL: Discard the Old Implementation

The file `static/SCC/util/crosshair.js` must be **deleted and rewritten from scratch**. Do not refactor it. Do not copy-paste from it. Do not let its structure influence yours.

The old design has architectural problems baked into its bones:

- DOM lookups (`document.getElementById`) inside the per-frame hot path
- Two `getBoundingClientRect` calls per frame (one in coordinate conversion, one in line positioning)
- `chartDiv.layout` read multiple times per frame
- `Plotly.relayout()` called synchronously during activation, blocking the main thread for 100-300ms
- Marker elements destroyed and recreated every time x changes (`container.innerHTML = ''`)
- Info panel rebuilt from HTML string every time x changes
- Linear search (`indexOf`) through unsorted-length arrays for data lookup
- Coordinate-to-pixel math duplicated across three functions

Read this spec. Build it clean.

What This Module Does

When the user holds **Shift** and moves the mouse over the chart:

1. **Crosshair lines** — gray dashed vertical + horizontal lines track cursor position
2. **Data markers** — semi-transparent shapes appear on data points at the current x-position
3. **Info panel** — sidebar shows date, cursor coordinates, and series values at current x

Releasing Shift hides everything and restores the previous sidebar state.

External Dependencies

```
import { chartState } from '../chartState.js';
import { CORRECTS, ERRORS } from '../config.js';
import { xPositionToDate } from './dates.js';
import { formatValue } from './format.js';
```

What these provide

- `chartState.traceStyles` — object containing style configs for each series. Structure:
 - `chartState.traceStyles.corrects.raw.seriesName` → display name (e.g., "Correct")
 - `chartState.traceStyles.corrects.raw.markerSize` → number
 - `chartState.traceStyles.errors.raw.seriesName` → display name
 - `chartState.traceStyles.errors.raw.textSize` → number (used as marker size for errors)
 - `chartState.traceStyles.timing.raw.seriesName` → display name
 - `chartState.traceStyles.timing.raw.markerSize` → number
 - `chartState.traceStyles.misc` — sub-object keyed by misc ID (e.g., `misc1`, `misc2`)
 - `chartState.traceStyles.misc.misc1.raw.seriesName` → display name
 - `chartState.traceStyles.misc.misc1.raw.markerSize` → number
- `CORRECTS`, `ERRORS` — string constants, series IDs
- `xPositionToDate(xRounded)` — converts integer x-position to a Date object
- `formatValue(number)` — formats a number for display (handles log scale values)

Exports required

```
javascript
export { init, activateCrosshair, deactivateCrosshair };
```

Same API surface as the old module. Drop-in replacement.

Chart Environment

- Plotly.js chart in `<div id="chart">`
 - X-axis: linear scale, integer positions
 - Y-axis: log10 scale
 - `chartDiv.layout` contains margins, axis ranges
 - `chartDiv.data` is an array of Plotly trace objects
 - Each trace has `.x` (sorted ascending array of numbers), `.y` (corresponding values), and `.meta` (`{ seriesName, aggType }`)
 - Traces with `seriesName` containing `'FloorShadow'` should be skipped
 - X data is sparse — not every integer position has a data point
-

Architecture: Two Update Tiers

Tier 1: Per-frame (60fps) — crosshair line position

Runs on every `requestAnimationFrame`. Does:

- Read `lastEvent.clientX` and `lastEvent.clientY` (no layout-forcing properties)
- Subtract cached rect offsets and margins (pure arithmetic)
- Boundary check against cached plot dimensions
- Two CSS `transform` writes: one for vertical line, one for horizontal line

Zero DOM reads. Zero layout triggers. Two style writes.

Tier 2: Per-x-change — data lookup, markers, info panel

Runs only when `Math.round(xValue)` differs from the previous frame. Does:

- Binary search each trace's x-array for the closest point
- Reposition pre-existing marker elements (show/hide, move via transform)
- Update pre-existing info panel text nodes (`.textContent` only)

No element creation. No innerHTML. No HTML string building.

Cache Strategy

What to cache and when to rebuild

A single `cache` object stores all values that would otherwise require DOM reads or Plotly layout access during the hot path.

Rebuild triggers:

- `activateCrosshair()` — full rebuild
- `window resize` event — full rebuild
- `plotly_relayout` event — full rebuild

Cache contents:

```
cache = {
  rect: DOMRect,           // from chartDiv.getBoundingClientRect()
  plotLeft: number,        // layout.margin.l
  plotTop: number,         // layout.margin.t
  plotWidth: number,       // rect.width - margin.l - margin.r
  plotHeight: number,      // rect.height - margin.t - margin.b
  plotBottom: number,      // margin.t + plotHeight
  xRange: [number, number], // layout.xaxis.range
  yRange: [number, number], // layout.yaxis.range
  xScale: number,          // plotWidth / (xRange[1] - xRange[0])
  yScale: number,          // plotHeight / (yRange[1] - yRange[0])
}
```

`xScale` and `yScale` let you convert data values to pixel offsets with a single multiply + add, avoiding repeated division.

Pixel from data value:

```
xPixel = plotLeft + (xValue - xRange[0]) * xScale
yPixel = plotBottom - (log10(yValue) - yRange[0]) * yScale
```

DOM Element Strategy

Created once at `init()` time

All DOM elements the module needs are created during `init()` and stored as direct references. Never looked up again via `getElementById` or `querySelector`.

Elements to create:

Element	ID	Purpose	Parent
Event overlay	<code>crosshair-event-overlay</code>	Captures mouse events when active	<code>#chart</code>
Line container	<code>crosshair-lines</code>	Contains both crosshair lines	<code>#chart</code>
Vertical line	<code>crosshair-v</code>	Vertical crosshair	<code>#crosshair-lines</code>
Horizontal line	<code>crosshair-h</code>	Horizontal crosshair	<code>#crosshair-lines</code>
Marker container	<code>crosshair-markers</code>	Contains data point markers	<code>#chart</code>
(Marker pool)	—	One div per series slot, reused	<code>#crosshair-markers</code>

All references stored in an `elements` object:

```
elements = {
  chart: HTMLElement,
  eventOverlay: HTMLElement,
  vLine: HTMLElement,
  hLine: HTMLElement,
  markerContainer: HTMLElement,
  markers: Map<string, HTMLElement>, // keyed by series ID
}
```

The `#chart` div must exist before `init()` is called. If it doesn't exist yet, `init()` should attach the keyboard listeners but defer DOM element creation to the first `activateCrosshair()` call. This is a lazy initialization pattern — set a `domReady` boolean, check it in `activate`, build if needed.

Marker pool

Pre-create one marker div per series type. There are at most ~8 series (corrects, errors, timing, misc1-misc5). Each marker is created once with its shape and color, then shown/hidden and repositioned via `transform` as needed.

When x changes, iterate the trace data results:

- If a series has data at this x: show its marker, set `transform: translate(xPx, yPx)`
- If a series has no data: hide its marker (`display: none`)

Never `innerHTML = "`. Never `appendChild` or `removeChild` in the hot path.

The marker pool can be built lazily — on the first activation, inspect `chartDiv.data` to see which traces exist, create a marker for each unique `seriesName`, and store them. If traces change later (user adds/removes series), rebuild the pool on next activation.

Info panel

The info panel lives inside `#crosshair-content` in the sidebar. Its structure:

```
#crosshair-info
├── .crosshair-section (Date)
│   ├── .crosshair-heading "Date"
│   ├── .crosshair-row → dayLabel (span.crosshair-value)
│   ├── .crosshair-row → monthLabel (span.crosshair-value)
│   └── .crosshair-row → yearLabel (span.crosshair-value)
├── .crosshair-section (Cursor)
│   ├── .crosshair-heading "Cursor"
│   ├── .crosshair-row → xLabel (span.crosshair-value)
│   └── .crosshair-row → yLabel (span.crosshair-value)
└── .crosshair-section (Series)
    ├── .crosshair-heading "Series"
    └── (series rows — built from pool, shown/hidden)
```

Pre-create this DOM structure at init. Store references to every `.crosshair-value` span. On update, set `.textContent` on the stored references. No HTML string construction, no `innerHTML`.

For the series section: pre-create one row per possible series (same pool as markers). Show/hide rows based on which series have data at current x. Update `.textContent` for name and value.

Activation / Deactivation

activateCrosshair()

Execute in this order:

1. Set `state.active = true`
2. Ensure DOM elements exist (lazy init if needed)
3. Rebuild cache (one `getBoundingClientRect()`, one `chartDiv.layout` read)
4. Attach `plotly_beforehover` handler that returns `false`
5. Set `eventOverlay.style.pointerEvents = 'auto'`
6. Attach `mousemove` listener to event overlay
7. Store current sidebar tab for later restoration
8. Show crosshair content panel, hide tabs

Do NOT call `Plotly.relayout()` to disable hovermode/dragmode. The combination of `plotly_beforehover → false` and the event overlay intercepting all mouse events is sufficient to prevent Plotly's hover computation. Calling `relayout` triggers Plotly's full 100-300ms recalculation pipeline, creating a noticeable delay on every Shift press.

If you absolutely must disable dragmode (to prevent accidental zoom initiation), defer it:

javascript

```
setTimeout(() => Plotly.relayout(chartDiv, { dragmode: false }), 0);
```

This moves it off the critical activation path. But test without it first — the event overlay should already prevent drag initiation.

deactivateCrosshair()

1. Set `state.active = false`
2. Cancel pending RAF
3. Remove `plotly_beforehover` handler
4. Set `eventOverlay.style.pointerEvents = 'none'`
5. Remove `mousemove` listener
6. Hide crosshair lines (`display: none`)
7. Hide all markers (`display: none`)
8. Hide crosshair content panel, restore tabs and previous active tab
9. Reset `lastXRounded`

If you deferred a `relayout` for dragmode, restore it here (also deferred to avoid blocking).

Mouse Movement Pipeline

mousemove on overlay

- store event reference as lastEvent
- if no RAF pending:
 - request RAF
 - set rafPending flag

RAF callback fires:

- clear rafPending flag
- read lastEvent.clientX, lastEvent.clientY
- compute xPixel = clientX - cache.rect.left
- compute yPixel = clientY - cache.rect.top
- boundary check: if outside plot area, hide lines and return
- compute xValue = cache.xRange[0] + (xPixel - cache.plotLeft) / cache.xScale
- compute yLogValue for display
- set vLine.style.transform = `translateX(\${xPixel}px)` ← note: pixel-space, not data-space
- set hLine.style.transform = `translateY(\${yPixel}px)`
- show lines if hidden
- xRounded = Math.round(xValue)
- if xRounded !== lastXRounded:
 - lastXRounded = xRounded
 - traceData = findTraceDataAtX(xRounded)
 - updateMarkers(xRounded, traceData)
 - updateInfoPanel(xRounded, yLogValue, traceData)

Important: line positioning is in pixel space

The vertical line's translateX should be in **pixel coordinates relative to the chart div**, not data coordinates. Same for horizontal. The lines are already positioned at `top: 0; left: 0` within the chart div, so the translate values are direct pixel offsets.

The vertical line needs:

- `transform: translateX(${xPixel}px)` — where xPixel is relative to chart div left edge
- `height` set to `cache.plotHeight` (set once on cache rebuild, not per frame)
- `top` offset to `cache.plotTop` (set once on cache rebuild)

Actually, a cleaner approach: position each line's initial offset via `top/left` (set once during cache rebuild to plotTop/plotLeft), then use `transform` for only the varying axis:

```
vLine: top = plotTop, left = 0, height = plotHeight
      per-frame: transform = translateX(${xPixel}px)

hLine: left = plotLeft, top = 0, width = plotWidth
      per-frame: transform = translateY(${yPixel}px)
```

This way the per-frame update is a single transform property per line.

Data Lookup: `findTraceDataAtX(xRounded)`

Iterates `chartDiv.data`. For each trace:

1. Skip if no `.meta`
2. Skip if `seriesName` includes `'FloorShadow'`
3. Binary search `.x` array for closest value to `xRounded`
4. If closest is within 0.5 of `xRounded` and the corresponding y value is not null/NaN, include it

Returns a Map keyed by `"seriesName-aggType"`, values are `{ seriesName, aggType, value }`.

Binary search implementation

The x arrays are sorted ascending. Find the insertion point for `xRounded`, then compare the elements immediately left and right of that point to find the closest.

javascript

```
function binarySearchClosest(arr, target) {
  let lo = 0, hi = arr.length - 1;
  if (hi < 0) return { index: -1, dist: Infinity };

  while (lo < hi) {
    const mid = (lo + hi) >> 1;
    if (arr[mid] < target) lo = mid + 1;
    else hi = mid;
  }

  let bestIdx = lo;
  let bestDist = Math.abs(arr[lo] - target);

  if (lo > 0) {
    const d = Math.abs(arr[lo - 1] - target);
    if (d < bestDist) {
      bestIdx = lo - 1;
      bestDist = d;
    }
  }

  return { index: bestIdx, dist: bestDist };
}
```

Use this instead of `indexOf`. This gives $O(\log n)$ per trace instead of $O(n)$.

Marker Styles

Series type	Color	Shape	Size source
corrects	 #22c55e (green)	circle	$\text{chartState.traceStyles[CORRECTS].raw.markerSize} \times 1.0$
errors	 #ef4444 (red)	circle	$\text{chartState.traceStyles[ERRORS].raw.textSize} \times 0.54$
timing	 #a855f7 (purple)	triangle-down	$\text{chartState.traceStyles.timing.raw.markerSize} \times 0.30$
misc*	 #f97316 (orange)	square	$\text{chartState.traceStyles.misc[id].raw.markerSize} \times 1.3$

All markers have 40% opacity. Add 8px padding to the computed visual size.

Shapes via CSS:

- circle:** `border-radius: 50%, background-color`
- square:** `no border-radius, background-color`
- triangle-down:** `width: 0; height: 0; border-left: Xpx solid transparent; border-right: Xpx solid transparent; border-top: Ypx solid <color>`

Series Name Resolution

To get the display name for a series:

```

javascript

function formatSeriesName(seriesId) {
  let config;
  if (seriesId && seriesId.startsWith('misc')) {
    config = chartState.traceStyles.misc?.[seriesId];
  } else if (seriesId) {
    config = chartState.traceStyles?.[seriesId];
  }

  if (config?.raw?.seriesName) return config.raw.seriesName;

  // Fallback
  const fallback = { corrects: 'Correct', errors: 'Incorrect', timing: 'Timing' };
  return fallback[seriesId] || seriesId;
}

```

Info Panel Content

Date section

Call `xPositionToDate(xRounded)` to get a Date object. Display:

Day: Wed | 15
Month: Jan | 01
Year: 2025

Cursor section

x: `<xRounded>`
y: `<formatValue(yValue)>`

Where `yValue` is `Math.pow(10, yLogValue)` — the actual y value, not the log.

Series section

For each entry in `traceData`:

- Display name from `formatSeriesName(seriesName)`
 - For timing series: display `formatValue(1 / value)` (reciprocal — timing is stored inverted)
 - For all others: display `formatValue(value)`
 - If `aggType !== 'raw'`, append `(aggType)` after the value
-

CSS Properties for Performance

Crosshair lines

```
css
position: absolute;
top: 0; /* overridden during cache rebuild to plotTop/plotLeft */
left: 0;
will-change: transform;
contain: layout paint;
pointer-events: none;
display: none; /* shown when active */
```

Vertical line: `width: 1px`, dashed gray via `repeating-linear-gradient(to bottom, gray 0px, gray 4px, transparent 4px, transparent 8px)`.

Horizontal line: `height: 1px`, dashed gray via `repeating-linear-gradient(to right, ...)`.

Line container

CSS

```
position: absolute;
top: 0; left: 0;
width: 100%; height: 100%;
pointer-events: none;
z-index: 50;
contain: strict;
```

Event overlay

CSS

```
position: absolute;
top: 0; left: 0;
width: 100%; height: 100%;
pointer-events: none; /* toggled to 'auto' when active */
z-index: 100;
```

Marker container

CSS

```
position: absolute;
top: 0; left: 0;
width: 100%; height: 100%;
pointer-events: none;
z-index: 51;
```

Event Listener Management

Attached at `init()` (permanent)

- `document` `keydown` — activate on Shift (guard against repeat with `shiftHeld` flag)
- `document` `keyup` — deactivate on Shift release
- `window` `blur` — deactivate (handles alt-tab while Shift held)

Attached at `activate()` (temporary)

- Event overlay `mousemove` — store event + request RAF
- `chartDiv` `plotly_beforehover` — return false
- `window` `resize` — rebuild cache (can debounce, 100-200ms is fine)
- `chartDiv` `plotly_relayout` — rebuild cache

Removed at `deactivate()`

- All four temporary listeners above

Store handler references in state so they can be removed cleanly.

State Object

javascript

```
const state = {
  active: false,
  shiftHeld: false,
  domReady: false,
  lastXRounded: null,
  rafPending: null,
  lastEvent: null,
  previousActiveTab: null,

  // Handler references for cleanup
  mouseMoveHandler: null,
  beforeHoverHandler: null,
  resizeHandler: null,
  relayoutHandler: null,

  // Cached geometry
  cache: null,

  // DOM references
  elements: null,
};
```

File Location

`static/SCC/util/crosshair.js` — same path as old file. Delete old, write new.

Summary of Performance Rules

1. **Zero DOM reads in the per-frame path.** All geometry is cached.
2. **Two style writes per frame** — one `transform` per crosshair line.
3. **No `Plotly.relayout()` in the activation path.** Defer or eliminate.
4. **No element creation/destruction during use.** Pool everything.
5. **No `innerHTML` during use.** Update `.textContent` on pre-built elements.
6. **No `getElementById` during use.** All references stored at init.
7. **Binary search for data lookup.** $O(\log n)$ per trace, not $O(n)$.
8. **Data features only run when x changes** — decoupled from frame rate.
9. `clientX`/`clientY` only — never `offsetX`/`offsetY` (those force synchronous layout).
10. **Cache invalidation on resize and relayout only** — not per frame.